

Operator Setup Guide

Weblo autonomous lead-gen pipeline — the one-time human setup before the build.

This is your follow-along checklist. It covers **only the things a human has to do** — creating accounts, spending money, and setting safety caps. The actual pipeline build is done by the AI build agent afterward (Phase 11).

How to use this: go top to bottom. Each step says *where to go*, *what to click*, *what to enter*, and *how to know it worked*. As you collect each secret value, paste it into the `.env` file in your project folder at the variable name in the blue → `.env` boxes.

Two notes before you start:

- Button labels and menu names on these websites change over time. If a label here doesn't match exactly, look for the closest match or use the site's search/help — the *flow* stays the same.
- The compliance/legal lines (CAN-SPAM, mailing address) are **general guidance, not legal advice** — confirm specifics with a qualified professional before you start real outreach.

Time: ~2 hours of work, spread across a day — Phase 3 (domain warm-up) and Phase 5 (mailbox) have waiting periods, so start those first.

Phase 1 — Lock down the Cloudflare token

You pasted a Cloudflare API token in chat earlier. Even though it's rolled, do this to be safe.

1. Go to <https://dash.cloudflare.com> and log in.
2. Top-right: **profile icon** → **My Profile** → **API Tokens** (or go straight to <https://dash.cloudflare.com/profile/api-tokens>).
3. If you see an **old** token you previously shared, click the ... next to it → **Delete** (or **Roll**). Keep only the current `weblo-pipeline` token.
4. Open `.env`. Confirm `CLOUDFLARE_API_TOKEN=` holds your **current** token, not the old one.
5. **OneDrive check:** your project is in `Downloads`. In File Explorer, right-click the `Site` folder — if you see OneDrive cloud/checkmark icons, your secrets may be syncing to the cloud. Move the folder to a non-synced location like `C:\weblo\`.

✅ **Done when:** only your current token exists in Cloudflare, and `.env` matches it.

Phase 2 — Confirm your domain is on Cloudflare

You already own **webloapp.com** (it's in your `.env`). Just confirm it.

1. At <https://dash.cloudflare.com>, look at the site list on the home page.
2. Confirm **webloapp.com** is listed. Click it → you land on its **Overview** page.
3. Click **DNS** in the left menu. You'll use this screen several times below (it's where you add the email-authentication records).

✓ **Done when:** you can see webloapp.com and open its DNS page.

Phase 3 — Google Workspace + email setup DO EARLY

This sets up your professional sending address (`you@webloapp.com`). The 2-week warm-up clock (Phase 4) can't start until this is done.

3a. Create the Workspace account

1. Go to <https://workspace.google.com> → **Start Free Trial**.
2. When asked about a domain, choose **"Use a domain I already own"** and enter **webloapp.com**.
3. Create your admin username — e.g. **austin@webloapp.com**. Set a strong password.
4. Finish signup. You land in the **Google Admin console** at <https://admin.google.com>.

→ **.env**: uncomment `SENDER_EMAIL=austin@webloapp.com` and `SENDER_NAME=Weblo` (EMAIL TRACK section).

3b. Verify you own the domain

1. The setup wizard asks you to verify via a **TXT record**, showing a value like `google-site-verification=xxxx`.
2. In Cloudflare → `webloapp.com` → **DNS** → **Records** → **Add record**: Type **TXT**, Name `@`, Content = the `google-site-verification=...` value → **Save**.
3. Back in Google, click **Verify**.

3c. Turn on email (MX records)

1. Google gives you **MX records**. In Cloudflare DNS, add each one: Type **MX**, Name `@`, Mail server = the value Google shows (e.g. `smtp.google.com`), Priority = the number shown → **Save**.
2. Back in Google, click **Activate Gmail** / **Continue**.

3d. Authenticate the domain (SPF, DKIM, DMARC) — critical for inbox delivery

Without these, cold email goes straight to spam. Add three TXT records in Cloudflare DNS:

- **SPF** — Type TXT, Name `@`, Content `v=spf1 include:_spf.google.com ~all`
- **DKIM** — In Admin console: **Apps** → **Google Workspace** → **Gmail** → **Authenticate email**. Click **Generate new record**, copy the host + value. In Cloudflare add: Type TXT, Name = host shown (e.g. `google._domainkey`), Content = the long `v=DKIM1...` value. Then in Google click **Start authentication**.
- **DMARC** — Type TXT, Name `_dmarc`, Content `v=DMARC1; p=none; rua=mailto:austin@webloapp.com`

3e. Verify it all passes

1. From `austin@webloapp.com`, send a test email to a personal Gmail.
2. In Gmail, open it → ⋮ → **Show original**. You want **SPF: PASS, DKIM: PASS, DMARC: PASS**.

✓ **Done when:** all three say PASS. (Doc 05 in your project folder has more detail if needed.)

Phase 4 — Start the email warm-up service LONG POLE

A brand-new domain has zero reputation. A warm-up service builds it over ~2 weeks while the rest gets built. Start it the same day as Phase 3.

1. Pick one: [Instantly](#), [Smartlead](#), or [Warmbox](#) (~\$30–50/mo).
2. Sign up → find the **"Warmup"** / **"Email Warmup"** section.
3. **Connect your mailbox:** choose **Google / Gmail**, sign in as `austin@webloapp.com`, grant access.
4. **Turn warmup ON.** Leave defaults. Let it run ~2 weeks.

✅ **Done when:** the service shows your mailbox "warming." **Do not send real cold email until ~2 weeks have passed** (the build agent enforces this too).

Phase 5 — Get a mailing address

US cold email legally needs a real physical postal address in the footer, and Lob needs a return address for postcards. Use a mailbox service, **not** your home address.

1. **Option A — The UPS Store:** <https://www.theupsstore.com> → **Mailboxes** → **Find a Location** → rent a mailbox (real street address, ~\$15–25/mo).
2. **Option B — USPS PO Box:** <https://www.usps.com/manage/po-boxes.htm> → reserve a box online.
3. Write the address in one line: `Name, 123 Main St #100, City, ST 12345`.

→ **.env:** `BUSINESS_MAILING_ADDRESS=` (one line, email footers) · `MAIL_FROM_NAME=Weblo` , `MAIL_FROM_LINE1=` , `MAIL_FROM_CITY=` , `MAIL_FROM_STATE=` , `MAIL_FROM_ZIP=` (split out, for Lob).

✅ **Done when:** you have a usable street address and it's in `.env` . (General guidance, not legal advice — confirm CAN-SPAM specifics with a professional.)

Phase 6 — Telegram bot

This is how the pipeline pings your phone when a lead is interested, and how you approve postcard batches.

1. On your phone, install **Telegram** and open it.
2. In search, type **@BotFather** and open the verified result (blue check).
3. Send `/newbot` . Give it a **name** (e.g. `Weblo Pipeline`) and a **username** ending in `bot` (e.g. `weblo_pipeline_bot`).
4. BotFather replies with a **token** like `8123456789:AAE...` . Copy it. → **.env:** `TELEGRAM_BOT_TOKEN=`
5. **Get your chat ID:** search your new bot, open it, tap **Start**, send any message ("hi").
6. On a computer, open this URL (paste your token for `<TOKEN>`):
`https://api.telegram.org/bot<TOKEN>/getUpdates`
In the JSON, find `"chat":{"id":123456789}` — that number is your chat ID. → **.env:** `TELEGRAM_CHAT_ID=`

✅ **Done when:** both `TELEGRAM_BOT_TOKEN` and `TELEGRAM_CHAT_ID` are in `.env` .

Phase 7 — Anthropic (Claude) account + key + spend cap 💰

This is the AI that writes the sample sites and emails. **The spend cap here is your #1 cost protection — do not skip it.**

1. Go to <https://console.anthropic.com> and sign in / create an account.
2. **Set a spending limit FIRST: Settings → Billing** (or **Limits / Usage limits**). Add a payment method, then set a **monthly spend limit** — start at **\$80** to match `CLAUDE_MONTHLY_BUDGET_USD` . This is the hard ceiling the vendor enforces even if code misbehaves.
3. **Create the API key: Settings → API Keys → Create Key**. Name it `weblo-pipeline` . Copy the `sk-ant-...` value — **shown only once**. → **.env:** `ANTHROPIC_API_KEY=`

✅ **Done when:** a monthly spend limit is set **and** the key is in `.env` .

Phase 8 — Google Cloud: lead-finding + server + cost caps 💰

Covers the Google Places API (finds businesses), its cost caps, and the free server that runs everything.

8a. Create the project

1. Go to <https://console.cloud.google.com>. Top bar: **project dropdown** → **New Project** → name `weblo-pipeline` → **Create**. Select it in the top bar.
2. Enable **billing**: search "Billing" in the top search bar → link a payment method.

8b. Enable Places API + get a key

1. Top search: "**Places API**" → open → **Enable**.
2. Left menu: **APIs & Services** → **Credentials** → **Create Credentials** → **API key**. Copy it.
→ `.env: GOOGLE_PLACES_API_KEY=`
3. Click the key → **API restrictions** → restrict to **Places API** → Save.

8c. Cap Places spending (do both)

1. **Hard quota cap**: **APIs & Services** → **Places API** → **Quotas & System Limits**. Find **Requests per day**, click the pencil, set a low daily cap (e.g. 1000). This actually stops spend.
2. **Budget alert**: top search "**Budgets & alerts**" → **Create Budget** → scope to this project → amount **\$20** (matches `PLACES_MONTHLY_BUDGET_USD`) → alerts at 50% / 90% / 100% → Finish.

8d. Create the free server (always-on machine)

1. Top search: "**Compute Engine**" → **Create Instance**.
2. Set: **Name** `weblo-pipeline` · **Region** `us-central1` (free-tier region) · **Machine type** `e2-micro` · **Boot disk** → Change → **Debian 12, 30 GB, Standard persistent disk** · leave HTTP/HTTPS **unchecked** → **Create**.
3. **\$1 safety alert**: Budgets & alerts → Create Budget → **\$1** → catches any accidental paid resource (the e2-micro itself is free).

✅ **Done when**: Places key in `.env`, a daily quota + \$20 budget set, and the `weblo-pipeline` VM shows a green "running" dot. (The build agent configures the software on this VM later — see doc 07.)

Phase 9 — Lob (postcards)

For mailing sample-site postcards to leads with no email.

1. Go to <https://dashboard.lob.com> → sign up.
2. Find **Settings** → **API Keys**. You'll see a **Test** key and a **Live** key.
→ `.env`: `LOB_API_KEY_TEST=` and `LOB_API_KEY_LIVE=`
3. Leave `LOB_MODE=test` for now — nothing real mails until you flip it to `live` after approving a batch.
`MAIL_MONTHLY_BUDGET_USD=100` + per-batch Telegram approval keep this gated.

✓ **Done when:** both Lob keys are in `.env`.

Phase 10 — Final .env check

Open `.env` and confirm every blank below is filled (email-track ones can stay commented until warm-up finishes).

Active now (postcard track + core):

- `ANTHROPIC_API_KEY` ✓ (Phase 7)
- `GOOGLE_PLACES_API_KEY` ✓ (Phase 8)
- `CLOUDFLARE_API_TOKEN` ✓ (Phase 1) · `CLOUDFLARE_ACCOUNT_ID` (the hex string in the Cloudflare URL after `dash.cloudflare.com/`)
- `TELEGRAM_BOT_TOKEN` , `TELEGRAM_CHAT_ID` ✓ (Phase 6)
- `CAMPAIGN_LOCATION` (e.g. `Dallas, TX`) , `CAMPAIGN_NICHE` (e.g. `plumbers`) — your first target
- `LOB_API_KEY_TEST` , `LOB_API_KEY_LIVE` ✓ (Phase 9)
- `MAIL_FROM_*` , `BUSINESS_MAILING_ADDRESS` ✓ (Phase 5)
- Budgets are pre-filled (`PIPELINE_MONTHLY_BUDGET_USD=150` , etc.) — adjust if you want.

Commented until email warm-up finishes (~2 weeks): `SENDER_EMAIL` , `SENDER_NAME` — fill but leave commented per the file's structure until the build agent turns on email.

✓ **Done when:** no required blank remains in the active section.

Phase 11 — Hand off to the build agent

You're done with the human part. Point the AI build agent at your project folder and tell it:

"Build the pipeline following the order of operations in `00-system-overview.md`. The `.env` is filled, the GCP VM exists, and the cost caps are set. Start with doc 07 (configure the VM), then doc 04, then the rest."

The agent provisions software on your VM, builds each stage, and wires in the cost guardrails (doc 08). It **cannot** spend beyond your Phase 7/8 caps, and **won't** send real cold email until your domain has warmed ~2 weeks.

Quick reference — where each thing lives

What	Where	Fills in .env
Cloudflare token / DNS	dash.cloudflare.com	CLOUDFLARE_*
Email address + auth	workspace.google.com / admin.google.com	SENDER_EMAIL , SENDER_NAME
Warm-up	instantly.ai / smartlead.ai	—
Mailing address	theupsstore.com / usps.com	MAIL_FROM_* , BUSINESS_MAILING_ADDRESS
Phone alerts	Telegram → @BotFather	TELEGRAM_*
Claude AI + cap	console.anthropic.com	ANTHROPIC_API_KEY
Lead-finding + server + caps	console.cloud.google.com	GOOGLE_PLACES_API_KEY
Postcards	dashboard.lob.com	LOB_API_KEY_*